

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



**BÁO CÁO KỸ THUẬT
XÂY DỰNG TRÒ CHƠI CARO AI**

Môn học: Cơ sở trí tuệ nhân tạo

Thành viên thực hiện

Nguyễn Duy Khánh 24022361

Nguyễn Sơn Duy 24022313

Ngày nộp: Ngày 16 tháng 5 năm 2026

HÀ NỘI – 2026

Mục lục

1	Mô tả bài toán cờ Caro	4
1.1	Giới thiệu bài toán	4
1.2	Lý do lựa chọn kích thước bàn cờ 9×9	4
1.3	Mục tiêu kỹ thuật	5
1.4	Thử thách về không gian trạng thái	5
1.5	Tùy chỉnh độ khó, hệ thống cấp độ và đối tượng tương tác	5
2	Luật chơi và điều kiện thắng	6
2.1	Quy tắc cơ bản	6
2.2	Điều kiện thắng cuộc	6
2.3	Trạng thái kết thúc khác	7
3	Cách biểu diễn trạng thái bàn cờ	7
3.1	Cấu trúc dữ liệu cốt lõi	7
3.2	Ký hiệu và ý nghĩa trạng thái	8
3.3	Quản lý lịch sử và trạng thái phụ	8
3.4	Biểu diễn hình ảnh (Giao diện người dùng)	9
3.5	Cấu trúc hằng số và quản lý cấu hình	9
3.6	Sơ đồ cấu trúc lớp và mối quan hệ hệ thống	10
3.7	Nguyên lý ánh xạ tọa độ	10
4	Cách sinh nước đi hợp lệ	11
4.1	Chiến lược thu hẹp không gian tìm kiếm	11
4.2	Quy trình sinh nước đi đầu tiên	11
4.3	Thuật toán quét vùng lân cận	11
4.4	Lợi ích kỹ thuật	12
4.5	Đánh giá hiệu năng	12
5	Cách kiểm tra trạng thái kết thúc	13

5.1	Cơ chế kiểm tra điều kiện thắng (<code>check_win</code>)	13
5.2	Phân tích độ phức tạp thuật toán	13
5.3	Kiểm tra trạng thái đầy bàn cờ (<code>is_board_full</code>)	14
5.4	Quản lý trạng thái logic ván đấu	14
5.5	Ý nghĩa của việc kiểm tra liên tục	14
5.6	Xử lý ngoại lệ và toàn vẹn dữ liệu	15
6	Thuật toán Minimax đã cài đặt	15
6.1	Tư tưởng thuật toán	15
6.2	Cách thức cài đặt	15
7	Thuật toán Alpha-Beta đã cài đặt	17
7.1	Giới thiệu cải tiến	17
7.2	Chi tiết cài đặt	17
7.3	Cơ chế cắt nhánh (Pruning)	19
7.4	Kỹ thuật sắp xếp nước đi (Move Ordering)	19
8	Hàm đánh giá trạng thái	20
8.1	Kỹ thuật Cửa sổ trượt (Sliding Window)	20
8.2	Hệ thống tính điểm (Heuristic Scoring)	21
8.3	Chiến thuật ưu tiên phòng thủ	21
9	Thiết kế các trạng thái thử nghiệm	24
10	Bảng kết quả thực nghiệm	24
11	Nhận xét về kết quả thực nghiệm	26
11.1	Tính chính xác của Alpha-Beta	26
11.2	Khả năng cắt giảm trạng thái (Nodes Visited)	26
11.3	Ảnh hưởng của độ sâu đến thời gian chạy	26
11.4	Ảnh hưởng của độ sâu đến chất lượng nước đi	28

12 Ưu điểm và Hướng cải tiến	28
12.1 Ưu điểm	28
12.2 Hạn chế và Hướng cải tiến	30

1 Mô tả bài toán cờ Caro

1.1 Giới thiệu bài toán

Cờ Caro (hay còn gọi là Gomoku) là một trò chơi chiến thuật đối kháng trực tiếp trên bàn cờ lưới giữa hai người chơi. Trong phạm vi dự án này, bài toán được giới hạn trên không gian bàn cờ có kích thước 9×9 ô vuông.

Về mặt lý thuyết trò chơi, đây là một dạng bài toán:

- **Perfect Information(Thông tin hoàn hảo):** Mọi trạng thái của bàn cờ, các nước đi trước đó và vị trí quân cờ hiện tại đều hiển thị rõ ràng cho cả hai bên, không có yếu tố ngẫu nhiên hay ẩn giấu.
- **Zero-sum Game(Trò chơi có tổng bằng không):** Lợi thế của người chơi này chính là bất lợi của người chơi kia. Kết quả cuối cùng chỉ có thể là thắng, thua hoặc hòa.

1.2 Lý do lựa chọn kích thước bàn cờ 9×9

Việc lựa chọn kích thước bàn cờ 9×9 (tương ứng với 81 ô cờ) không phải là ngẫu nhiên, mà là một sự tính toán kỹ lưỡng dựa trên các yếu tố sau:

- **Cân bằng giữa độ khó và không gian trạng thái:** Với kích thước 9×9 , không gian trạng thái của trò chơi đủ lớn để tạo ra những thử thách chiến thuật phức tạp cho người chơi. Tuy nhiên, nó vẫn nằm trong giới hạn cho phép để các thuật toán tìm kiếm như Minimax có thể duyệt đến độ sâu cần thiết nhằm đưa ra nước đi thông minh.
- **Hiệu năng tính toán trên ngôn ngữ Python:** Dự án được phát triển dựa trên ngôn ngữ Python – một ngôn ngữ thông dịch có tốc độ thực thi thường chậm hơn so với các ngôn ngữ biên dịch như C++ hay C#.
- **Tối ưu hóa thời gian phản hồi:** Nếu sử dụng bàn cờ chuẩn 15×15 hoặc 19×19 , số lượng nhánh tại mỗi nút lá trong cây quyết định sẽ bùng nổ theo hàm mũ (N^2), dẫn đến hiện tượng "trễ" (lag) khi AI tính toán. Kích thước 9×9 giúp hệ thống duy trì thời gian phản hồi dưới thấp cho mỗi nước đi của AI, đảm bảo trải nghiệm người dùng mượt mà mà không cần đến các cấu hình phần cứng quá mạnh.

1.3 Mục tiêu kỹ thuật

Mục tiêu cốt lõi của dự án là xây dựng một tác tử thông minh (AI Agent) có khả năng tự động hóa quá trình ra quyết định thông qua:

- **Phân tích trạng thái:** Đánh giá giá trị chiến thuật của cấu hình quân cờ hiện tại trên bàn cờ.
- **Tối ưu hóa nước đi:** Tìm kiếm trong không gian trạng thái để chọn ra nước đi dẫn đến khả năng chiến thắng cao nhất hoặc ngăn chặn đối thủ giành thắng lợi.
- **Cân bằng chiến thuật:** Đảm bảo AI có khả năng chuyển đổi linh hoạt giữa trạng thái tấn công (tạo chuỗi quân) và phòng thủ (chặn chuỗi quân của đối thủ).

1.4 Thử thách về không gian trạng thái

Mặc dù kích thước bàn cờ 9×9 đã được thu nhỏ so với bàn cờ chuẩn, số lượng trạng thái tổ hợp vẫn rất lớn. Điều này đặt ra yêu cầu về việc áp dụng các thuật toán tìm kiếm và kỹ thuật tối ưu hóa:

- **Thuật toán Minimax:** Duyệt cây quyết định để giả định các nước đi phản ứng của đối thủ.
- **Cắt tỉa Alpha-Beta:** Kỹ thuật tối ưu hóa nhằm loại bỏ các nhánh không tiềm năng, giúp tăng tốc độ xử lý mà không làm mất đi tính chính xác của quyết định.
- **Hàm lượng giá Heuristic:** Xây dựng thang điểm để định lượng giá trị các thế cờ khi cây tìm kiếm không thể duyệt đến trạng thái kết thúc cuối cùng.

1.5 Tùy chỉnh độ khó, hệ thống cấp độ và đối tượng tương tác

Hệ thống được thiết kế linh hoạt nhằm phục vụ các mục đích sử dụng và trình độ người chơi khác nhau thông qua trình đơn chọn độ khó (`draw_difficulty_menu`). Trí thông minh của tác tử AI được điều chỉnh trực tiếp thông qua tham số độ sâu tìm kiếm (`ai_depth`), cụ thể:

- **Mức Dễ (Easy):** Thiết lập độ sâu tìm kiếm bằng 2. Ở mức này, AI chỉ có khả năng tính toán các nước đi phản kháng đơn giản trước mắt, phù hợp cho người chơi mới làm quen với luật chơi.

- **Mức Trung bình (Medium):** Thiết lập độ sâu tìm kiếm bằng 3. Đây là mức độ cân bằng nhất, AI có thể nhận diện được các thế cờ hiểm và đưa ra các nước đi có tính chiến thuật trong khi vẫn duy trì thời gian phản hồi cực nhanh (thường dưới 1 giây).
- **Mức Khó (Hard):** Thiết lập độ sâu tìm kiếm bằng 4. AI khai thác tối đa sức mạnh của thuật toán Alpha-Beta để duyệt qua hàng chục nghìn trạng thái, cho phép phát hiện các cạm bẫy phức tạp và các thế "gài" từ xa. Đây là thử thách thực sự cho những người chơi có kinh nghiệm.

Ngoài ra, hệ thống còn hỗ trợ chế độ **Người chơi vs Người chơi (Human vs Human)**, cho phép sử dụng chương trình như một bàn cờ số hóa trực quan.

2 Luật chơi và điều kiện thắng

2.1 Quy tắc cơ bản

Trò chơi tuân thủ các quy tắc đối kháng cơ bản của cờ Caro truyền thống, được thiết lập trong mã nguồn thông qua lớp `CaroGame`:

- **Phân định lượt đi:** Hai người chơi được định danh bằng hai ký hiệu là X và O . Trong đó, người chơi X (thường là con người) luôn được quyền thực hiện nước đi đầu tiên để đảm bảo tính nhất quán của hệ thống.
- **Cơ chế đặt quân:** Tại mỗi lượt, người chơi chỉ được phép đặt duy nhất một quân cờ của mình vào một ô còn trống (được biểu diễn bằng ký tự ' ') trên bàn cờ lưới 9×9 .

2.2 Điều kiện thắng cuộc

Hệ thống xác định người thắng cuộc dựa trên hàm kiểm tra `check_win(player)`. Điều kiện thắng trong phiên bản này được tùy chỉnh như sau:

- **Cấu trúc chiến thắng:** Một người chơi sẽ giành chiến thắng ngay lập tức nếu tạo được một đường thẳng liên tục gồm ít nhất **4 quân cờ** cùng loại.
- **Các hướng kiểm tra:** Thuật toán thực hiện quét theo 4 trục không gian chính xuất phát từ vị trí quân cờ vừa đánh:

1. *Hàng ngang*: Kiểm tra các ô liên tiếp theo trục tung $(0, 1)$.
2. *Hàng dọc*: Kiểm tra các ô liên tiếp theo trục hoành $(1, 0)$.
3. *Đường chéo chính*: Kiểm tra theo hướng từ trên trái xuống dưới phải $(1, 1)$.
4. *Đường chéo phụ*: Kiểm tra theo hướng từ trên phải xuống dưới trái $(1, -1)$.

2.3 Trạng thái kết thúc khác

Ngoài việc phân định thắng thua, ván đấu có thể kết thúc trong trường hợp sau:

- **Trạng thái hòa (Draw)**: Xảy ra khi hàm `is_board_full()` trả về kết quả đúng, đồng nghĩa với việc toàn bộ 81 ô trên bàn cờ đã được lấp đầy nhưng không có người chơi nào tạo được chuỗi chiến thắng.
- **Dừng trận đấu chủ động**: Người chơi có thể lựa chọn thoát ra Menu chính thông qua hộp thoại xác nhận (`CONFIRM_QUIT`) để kết thúc ván đấu bất cứ lúc nào.

3 Cách biểu diễn trạng thái bàn cờ

3.1 Cấu trúc dữ liệu cốt lõi

Hệ thống quản lý trò chơi dựa trên sự kết hợp giữa lưu trữ trạng thái vật lý và bộ nhớ lượng giá tức thời:

- **`self.board`**: Một ma trận hai chiều kích thước 9×9 . Mỗi phần tử đại diện cho một ô trên bàn cờ, nhận một trong ba giá trị:

- ‘X’: Người chơi
- ‘O’: Máy
- ‘.’: Ô trống

Đây là dữ liệu gốc để truy xuất trạng thái hiển thị và kiểm tra điều kiện thắng/thua.

- **`self.current_board_score`**: Biến số nguyên lưu trữ tổng điểm lượng giá của toàn bộ bàn cờ tại trạng thái hiện hành.
 - **Khởi tạo**: Giá trị này được xác định một lần duy nhất khi bắt đầu ván đấu thông qua hàm đánh giá tổng quát `evaluate_board()`.

- **Cập nhật (Incremental Update):** Trong quá trình duyệt cây tìm kiếm, thay vì tính toán lại toàn bộ bàn cờ với độ phức tạp $O(N^2)$, hệ thống sử dụng phương pháp cập nhật hiệu số. Cụ thể, tổng điểm sẽ được cộng dồn hoặc trừ đi phần chênh lệch giữa điểm cũ và điểm mới tại vùng lân cận của nước đi vừa thực hiện ($O(1)$).
- **Vai trò:** Cấu trúc này cho phép hàm đánh giá trạng thái đạt tốc độ phản hồi gần như tức thời, triệt tiêu gánh nặng tính toán lặp lại tại các nút lá của thuật toán Minimax và Alpha-Beta.

3.2 Ký hiệu và ý nghĩa trạng thái

Hệ thống sử dụng các hằng số ký tự đơn giản để định nghĩa các thực thể tham gia vào bàn cờ, giúp giảm thiểu bộ nhớ và tăng tốc độ so sánh trong thuật toán:

- **Ký tự 'X':** Đại diện cho quân cờ của người chơi (hoặc người chơi thứ nhất trong chế độ hai người).
- **Ký tự 'O':** Đại diện cho quân cờ của máy (AI) hoặc người chơi thứ hai. Đây cũng là đối tượng mục tiêu để hàm lượng giá AI tính toán điểm số tối ưu.
- **Lượt đi hiện tại (self.current_player):** Biến lưu trữ ký tự của người chơi đến lượt, cho phép hệ thống kiểm soát logic luân phiên nước đi một cách chính xác.

3.3 Quản lý lịch sử và trạng thái phụ

Để hỗ trợ các tính năng nâng cao và kiểm soát luồng trò chơi, các cấu trúc bổ trợ sau được triển khai:

- **Danh sách lịch sử (self.history):** Lưu trữ một ngăn xếp (*stack*) chứa các bộ tọa độ (r, c) của từng nước đi. Đây là cơ sở dữ liệu duy nhất để thực hiện chức năng Hoàn tác (*Undo*).
- **Trạng thái trò chơi (self.state):** Định nghĩa các ngữ cảnh khác nhau của ứng dụng như 'MENU' (Màn hình chính), 'PLAYING' (Đang thi đấu), và 'CONFIRM_QUIT' (Xác nhận thoát).
- **Biến cờ hiệu (self.game_over):** Một giá trị logic (*Boolean*) dùng để đóng băng bàn cờ khi ván đấu đã phân định thắng thua hoặc hòa.

Một điểm nổi bật trong trải nghiệm người dùng là tính năng Hoàn tác được xây dựng với logic xử lý thông minh để đảm bảo tính công bằng và toàn vẹn dữ liệu:

- **Giới hạn quyền lực:** Để tránh việc lạm dụng, hệ thống giới hạn mỗi ván đấu người chơi chỉ được phép Hoàn tác tối đa 2 lần thông qua biến `undo_count`.
- **Cơ chế chống lặp (Anti-abuse):** Biến cờ hiệu `just_undid` được sử dụng để ngăn chặn việc người chơi thực hiện Hoàn tác liên tiếp nhiều lần trong một lượt đi. Quyền Hoàn tác chỉ được mở lại sau khi người chơi thực hiện một nước đi mới.
- **Logic thích ứng theo chế độ chơi:**
 - *Trong chế độ 2 người:* Mỗi lệnh Undo chỉ lùi lại duy nhất 1 nước đi vừa thực hiện.
 - *Trong chế độ đấu với AI:* Khi người chơi yêu cầu Hoàn tác, hệ thống tự động loại bỏ 2 nước đi gần nhất (bao gồm 1 nước của AI và 1 nước của người chơi). Điều này giúp đưa bàn cờ về đúng trạng thái trước khi người chơi thực hiện sai lầm, thay vì chỉ xóa nước đi của máy.
- **Khôi phục trạng thái logic:** Sau khi Hoàn tác, các biến trạng thái quan trọng như `game_over`, `winner`, và `winning_line` đều được đặt lại về giá trị mặc định, cho phép ván đấu tiếp tục diễn ra bình thường ngay cả khi người chơi vừa thực hiện Hoàn tác sau khi đã có kết quả thắng/thua.

3.4 Biểu diễn hình ảnh (Giao diện người dùng)

Trạng thái logic từ ma trận được ánh xạ sang giao diện đồ họa thông qua thư viện Pygame:

- **Tọa độ đồ họa:** Vị trí của một quân cờ tại hàng r , cột c được xác định dựa trên kích thước ô vuông (`CELL_SIZE`) và tọa độ gốc của bàn cờ (`BOARD_X`, `BOARD_Y`).
- **Hình ảnh trực quan:** Quân X được vẽ bằng hai đường chéo đan xen màu đỏ, trong khi quân O được biểu diễn bằng một đường tròn màu xanh lục, giúp người chơi dễ dàng phân biệt.

3.5 Cấu trúc hằng số và quản lý cấu hình

Để đảm bảo tính linh hoạt và dễ bảo trì, các tham số cốt lõi của trò chơi được định nghĩa tập trung thông qua hệ thống hằng số. Việc này giúp việc thay đổi quy mô bàn cờ hoặc

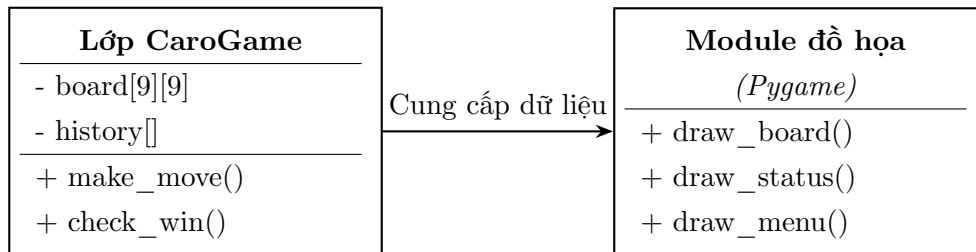
giao diện có thể thực hiện nhanh chóng mà không cần can thiệp sâu vào logic xử lý.

Bảng 1: Các hằng số cấu hình hệ thống

Hằng số	Giá trị	Ý nghĩa
SIZE	9	Kích thước ma trận bàn cờ (9×9)
CELL_SIZE	50	Kích thước của mỗi ô vuông (pixel)
BOARD_SIZE	SIZE * CELL_SIZE	Tổng kích thước khu vực chơi
X_COLOR	(200, 50, 50)	Màu đỏ đặc trưng cho quân X
O_COLOR	(50, 50, 200)	Màu xanh đặc trưng cho quân O
BOARD_BG	(222, 184, 135)	Màu gỗ nền của bàn cờ

3.6 Sơ đồ cấu trúc lớp và mối quan hệ hệ thống

Kiến trúc phần mềm của dự án được tách biệt rõ ràng giữa tầng **Logic trò chơi** (Back-end) và tầng **Hiển thị đồ họa** (Front-end). Mối quan hệ này được thể hiện qua sơ đồ dưới đây:



Hình 1: Mối quan hệ giữa Logic xử lý và Module hiển thị

3.7 Nguyên lý ánh xạ tọa độ

Sự kết nối giữa logic và đồ họa được thực hiện qua công thức ánh xạ điểm. Khi người dùng tương tác với màn hình tại tọa độ (x, y) , hệ thống chuyển đổi thành chỉ số hàng r và cột c trong ma trận:

$$r = \lfloor \frac{y - BOARD_Y}{CELL_SIZE} \rfloor, \quad c = \lfloor \frac{x - BOARD_X}{CELL_SIZE} \rfloor$$

Quá trình này đảm bảo tính chính xác tuyệt đối giữa thao tác vật lý và dữ liệu bên trong của AI.

4 Cách sinh nước đi hợp lệ

4.1 Chiến lược thu hẹp không gian tìm kiếm

Trong trò chơi Caro, không gian trạng thái có thể bùng nổ rất nhanh theo độ sâu của cây tìm kiếm. Với bàn cờ 9×9 , tổng số ô là 81, nếu xét mọi ô trống, số lượng nhánh tại mỗi nút sẽ vô cùng lớn. Để giải quyết vấn đề này, hàm `get_valid_moves()` triển khai chiến lược **Vùng lân cận (Adjacent Search)**:

- Hệ thống chỉ ưu tiên xem xét các ô trống nằm trong bán kính lân cận trực tiếp (8 ô xung quanh) của các quân cờ đã có trên bàn cờ.
- Việc này dựa trên đặc điểm thực tế của cờ Caro: một nước đi hiệu quả thường phải có sự kết nối với các quân cờ hiện tại để tạo chuỗi hoặc ngăn chặn đối thủ.

4.2 Quy trình sinh nước đi đầu tiên

Do tại thời điểm bắt đầu ván đấu, bàn cờ hoàn toàn trống và không có "vùng lân cận" để xác định, thuật toán áp dụng logic mặc định:

- AI sẽ kiểm tra nếu bàn cờ chưa có bất kỳ quân cờ nào thông qua biến `has_piece`.
- Trong trường hợp này, AI tự động chọn vị trí trung tâm bàn cờ tại tọa độ $(\lfloor \frac{SIZE}{2} \rfloor, \lfloor \frac{SIZE}{2} \rfloor)$ để sinh nước đi duy nhất.
- Việc chiếm vị trí trung tâm là một chiến thuật tối ưu để tối đa hóa khả năng phát triển các hướng tấn công ngay từ đầu ván.

4.3 Thuật toán quét vùng lân cận

Đối với các giai đoạn sau của ván đấu, quy trình sinh nước đi diễn ra như sau:

1. Duyệt qua toàn bộ ma trận 9×9 .
2. Đối với mỗi ô trống ('.'), thuật toán thực hiện một vòng lặp phụ để quét 8 vị trí xung quanh nó (trên, dưới, trái, phải và 4 đường chéo).
3. Nếu phát hiện ít nhất một ô xung quanh có chứa quân cờ ('X' hoặc 'O'), ô trống đó sẽ được coi là một "nước đi tiềm năng" và thêm vào danh sách hợp lệ.

4. Sử dụng các biến cờ hiệu (**adjacent**) để tối ưu tốc độ duyệt và thoát vòng lặp ngay khi tìm thấy quân cờ lân cận.
5. Mô tả tọa độ: Nếu một quân cờ đang tồn tại ở vị trí tọa độ (r, c) , các ô trống tiềm năng được thuật toán đưa vào danh sách xét duyệt sẽ nằm trong phạm vi giới hạn bởi khoảng hàng $[r - 1, r + 1]$ và khoảng cột $[c - 1, c + 1]$.
6. Quy trình quét 8 ô xung quanh: Với mỗi ô trống, thuật toán kiểm tra sự tồn tại của quân cờ tại các vị trí $(r + dr, c + dc)$ với $dr, dc \in \{-1, 0, 1\}$, loại trừ trường hợp $(0, 0)$.

4.4 Lợi ích kỹ thuật

Cách tiếp cận này mang lại những ưu điểm vượt trội cho hiệu suất của AI:

- **Giảm số lượng nhánh (Branching Factor):** Số lượng nước đi cần xét tại mỗi bước giảm đáng kể, giúp thuật toán Minimax và Alpha-Beta duyệt được sâu hơn trong cùng một khoảng thời gian.
- **Tập trung tính toán:** AI không lãng phí tài nguyên vào các ô ở quá xa hoặc các vị trí cô lập không có giá trị chiến thuật ngay lập tức.
- **Tăng trải nghiệm người dùng:** Giảm thiểu thời gian chờ đợi phản hồi từ AI, giúp trò chơi diễn ra mượt mà và tự nhiên hơn.

4.5 Đánh giá hiệu năng

Kỹ thuật sinh nước đi theo vùng lân cận mang lại những cải tiến vượt trội về mặt toán học và trải nghiệm người dùng:

- **Tối ưu hóa không gian tìm kiếm:** Kỹ thuật này giúp giảm độ phức tạp không gian tìm kiếm từ $O(N^2)$ xuống còn $O(k)$, với k là số ô trống bao quanh các quân cờ hiện có trên bàn cờ. Điều này tối ưu hóa đáng kể cho các thuật toán tìm kiếm sâu ở các phần sau.
- **Tập trung tài nguyên:** AI không lãng phí tài nguyên tính toán vào các ô ở quá xa hoặc các vị trí cô lập không có giá trị chiến thuật ngay lập tức.
- **Tốc độ phản hồi:** Giảm thiểu thời gian xử lý của hàm `bot_move()`, đảm bảo AI phản hồi nhanh chóng kể cả khi tăng độ sâu tìm kiếm (Depth) lên mức cao.

5 Cách kiểm tra trạng thái kết thúc

5.1 Cơ chế kiểm tra điều kiện thắng (check_win)

Hệ thống sử dụng một thuật toán duyệt theo hướng để xác định liệu một người chơi đã đạt được điều kiện thắng hay chưa ngay sau mỗi nước đi:

- **Xác định hướng duyệt:** Thuật toán định nghĩa 4 cặp vector hướng tương ứng với các trục đối xứng của một ô cờ: ngang $(0, 1)$, dọc $(1, 0)$, chéo chính $(1, 1)$ và chéo phụ $(1, -1)$.
- **Đếm quân liên tiếp:** Tại mỗi ô có chứa quân cờ của người chơi hiện tại, chương trình sẽ thực hiện lệnh lặp để đếm các quân cùng loại nằm liên tiếp nhau theo các hướng đã định nghĩa.
- **Ngưỡng xác lập:** Nếu biến đếm (count) đạt giá trị bằng 4, hàm sẽ trả về kết quả khẳng định chiến thắng và ghi nhận danh tính người thắng cuộc thông qua biến `self.winner`.

5.2 Phân tích độ phức tạp thuật toán

Trong cấu trúc tối ưu hóa, độ phức tạp tính toán của các hàm kiểm tra trạng thái và lượng giá được duy trì ở mức tối thiểu, đảm bảo hiệu năng hệ thống:

- **Kiểm tra điều kiện thắng (check_win):**

Đạt độ phức tạp $O(1)$ tại mỗi nước đi. Thay vì duyệt toàn bộ 81 ô cờ, thuật toán chỉ tập trung quét 4 trục không gian ngay tại tọa độ vừa thực hiện nước đi.

Do số lượng ô cần kiểm tra là hằng số và không phụ thuộc vào kích thước bàn cờ, thời gian thực thi luôn ổn định và cực nhanh.

- **Hàm lượng giá Heuristic (evaluate_board):**

Được cải tiến từ độ phức tạp $O(N^2)$ xuống còn $O(1)$ nhờ kỹ thuật cập nhật cộng dồn (*Incremental Update*).

- Nguyên lý:

Thay vì quét lại toàn bộ bàn cờ để tính điểm cho từng nút trong cây quyết định, hệ thống chỉ tính toán sự thay đổi điểm số tại vùng lân cận trực tiếp của nước đi mới thông qua hàm `get_local_score(r, c)`.

– **Hiệu quả:**

AI chỉ cần xem xét tối đa 16 dải ô tiềm năng xung quanh vị trí vừa đánh thay vì phải xử lý toàn bộ 81 ô. Điều này giúp triệt tiêu gánh nặng tính toán, cho phép thuật toán Alpha-Beta duyệt sâu hơn mà vẫn duy trì thời gian phản hồi dưới 1 giây ở các cấp độ khó.

5.3 Kiểm tra trạng thái đầy bàn cờ (`is_board_full`)

Để xử lý tình huống trận đấu kết thúc mà không có người thắng, hàm kiểm tra trạng thái lấp đầy được triển khai:

- **Nguyên lý hoạt động:** Thuật toán thực hiện hai vòng lặp lồng nhau để quét qua toàn bộ 81 ô của ma trận 9×9 .
- **Điều kiện dừng:** Nếu phát hiện bất kỳ ô nào vẫn mang giá trị ký tự ' ' (ô trống), hàm sẽ trả về `False` và trò chơi tiếp tục.
- **Xác lập kết quả hòa:** Trong trường hợp toàn bộ ma trận không còn ô trống và hàm `check_win` chưa được kích hoạt, hệ thống sẽ công bố kết quả hòa (`Draw`).

5.4 Quản lý trạng thái logic ván đấu

Khi một trong hai điều kiện trên được thỏa mãn, hệ thống sẽ thực hiện các thao tác đóng băng logic:

- **Kích hoạt biến `game_over`:** Chuyển giá trị biến cờ hiệu sang `True` để ngăn chặn mọi thao tác đặt quân tiếp theo của người dùng hoặc AI.
- **Cập nhật giao diện thông báo:** Dựa trên kết quả từ các hàm kiểm tra, hệ thống sẽ hiển thị các chuỗi văn bản tương ứng như: "Người chơi X Thắng!", "Máy (O) Thắng!", hoặc "Hòa!" ngay phía dưới bàn cờ.
- **Mở khóa các tùy chọn:** Chỉ sau khi trạng thái kết thúc được xác lập, nút "Chơi Lại" mới hiển thị để người dùng có thể khởi tạo lại bàn cờ thông qua hàm `reset_game`.

5.5 Ý nghĩa của việc kiểm tra liên tục

Việc kiểm tra trạng thái kết thúc ngay sau mỗi nước đi đảm bảo tính chính xác và kịp thời của trò chơi:

- Đảm bảo ván đấu dừng lại đúng thời điểm, tránh các tranh chấp về luật chơi.
- Cung cấp dữ liệu đầu vào cho AI để nó nhận diện được các trạng thái "lá" (thắng hoặc thua) trong cây tìm kiếm Minimax, từ đó đưa ra điểm số lượng giá cực đại hoặc cực tiểu chính xác.

5.6 Xử lý ngoại lệ và toàn vẹn dữ liệu

Để đảm bảo tính chính xác của dữ liệu bàn cờ và tránh các lỗi logic trong quá trình tương tác, hệ thống triển khai cơ chế kiểm soát chặt chẽ tại hàm `make_move`:

- **Kiểm tra ô đã có quân:** Hệ thống xử lý ngoại lệ khi người dùng click vào một ô đã tồn tại quân cờ bằng cách thực hiện câu lệnh điều kiện `if self.board[r][c] == '.'`.
- **Bảo vệ dữ liệu:** Nếu ô mục tiêu không trống, hàm sẽ lập tức trả về giá trị `False` và không thực hiện bất kỳ thay đổi nào lên ma trận bàn cờ hay danh sách lịch sử.
- **Đảm bảo logic:** Cơ chế này giúp ngăn chặn việc ghi đè quân cờ, đảm bảo mỗi ô chỉ chứa duy nhất một giá trị đại diện xuyên suốt ván đấu, từ đó duy trì tính toàn vẹn cho dữ liệu đầu vào của AI.

6 Thuật toán Minimax đã cài đặt

6.1 Tư tưởng thuật toán

Minimax là thuật toán tìm kiếm có đối thủ, duyệt qua không gian trạng thái dưới dạng một cây quyết định. Thuật toán giả định rằng cả hai người chơi đều chơi tối ưu. Trong cấu trúc trò chơi này:

- **MAX (Má - O):** Cố gắng tối đa hóa điểm số của trạng thái bàn cờ.
- **MIN (Người - X):** Cố gắng tối thiểu hóa điểm số của trạng thái bàn cờ.

6.2 Cách thức cài đặt

Thuật toán được triển khai thông qua hàm đệ quy `minimax(depth, is_maximizing)`, tích hợp quy trình quản lý điểm số tối ưu (*Incremental Update*) thay vì quét lại toàn bộ bàn cờ.

Tại mỗi nút đệ quy, hệ thống thực hiện các bước xử lý sau:

1. Kiểm tra trạng thái kết thúc (Base cases):

Hệ thống kiểm tra xem có bên nào đạt điều kiện thắng, thua hoặc trạng thái hòa hay không.

- Nếu Máy thắng: Trả về điểm số cực đại

$$+10,000,000 + \text{depth}$$

- Nếu Người thắng: Trả về điểm số cực tiểu

$$-10,000,000 - \text{depth}$$

- Nếu Hòa: Trả về

$$0$$

2. Kiểm tra giới hạn độ sâu (Depth Limit):

Khi đạt đến độ sâu giới hạn ($\text{depth} == 0$), thay vì gọi lại hàm quét toàn bộ ma trận 9×9 (`evaluate_board`) gây tốn kém tài nguyên, thuật toán lập tức trả về giá trị hiện hành của biến:

`self.current_board_score`

3. Quy trình thử nghiệm nước đi và cập nhật điểm số cộng dồn:

Thuật toán lấy danh sách các nước đi hợp lệ (chỉ duyệt các ô trống lân cận quân cờ hiện có) và thực hiện quy trình “Thử – Sai” kết hợp quản lý hiệu số điểm:

(a) Lưu trữ điểm số cực bộ cũ:

Trước khi đặt quân, gọi hàm `get_local_score(r, c)` để ghi lại giá trị điểm của các dải ô bị tác động bởi tọa độ sắp đánh.

(b) Thực hiện nước đi giả lập:

Đặt quân cờ của người chơi hiện tại (`'0'` cho Máy hoặc `'X'` cho Người) vào ma trận bàn cờ.

(c) Cập nhật tổng điểm bàn cờ:

Tính điểm cục bộ mới tại vị trí vừa đánh. Tổng điểm của toàn bộ bàn cờ (`self.current_board_score`) được cập nhật ngay lập tức bằng cách cộng thêm phần chênh lệch giữa điểm mới và điểm cũ:

$$\text{score} = \text{score} + (\text{new_score} - \text{old_score})$$

(d) **Gọi đệ quy:**

Tiếp tục gọi hàm `minimax` cho các cấp độ sâu tiếp theo với lượt của đối thủ.

(e) **Hoàn tác trạng thái (Backtrack):**

Thu hồi quân cờ và trừ đi phần chênh lệch điểm đã cộng trước đó.

Bước này đảm bảo bàn cờ và tổng điểm quay về trạng thái gốc trước khi thử nghiệm các nhánh nước đi khác.

4. **Lựa chọn giá trị tối ưu:**

- Tại nút Máy (MAX): Thuật toán chọn nhánh mang lại điểm số lớn nhất từ các nhánh con.
- Tại nút Người (MIN): Thuật toán chọn nhánh mang lại điểm số nhỏ nhất (giả định người chơi luôn có nước phản kháng tối ưu nhất).

5. **Thống kê hiệu năng:**

Tích hợp biến đếm `self.nodes_visited` tăng thêm 1 đơn vị sau mỗi lần hàm được gọi.

Dữ liệu này được sử dụng để phân tích số lượng trạng thái đã duyệt trong phần thực nghiệm.

7 Thuật toán Alpha-Beta đã cài đặt

7.1 Giới thiệu cải tiến

Alpha-Beta Pruning (Cắt tỉa Alpha-Beta) là một phiên bản tối ưu hóa của Minimax. Thuật toán này không đánh giá các nhánh của cây tìm kiếm nếu phát hiện ra rằng nhánh đó chắc chắn sẽ mang lại kết quả tồi tệ hơn so với một nhánh đã được khám phá trước đó. Kết quả đầu ra của Alpha-Beta hoàn toàn giống với Minimax ở cùng một độ sâu, nhưng tốc độ thực thi nhanh hơn vượt trội.

7.2 Chi tiết cài đặt

Hàm `alphabeta(depth, alpha, beta, is_maximizing)` được xây dựng dựa trên cấu trúc đệ quy của thuật toán Minimax nhưng được tích hợp thêm các tham số điều hướng

và cơ chế quản lý điểm số cộng dồn (*Incremental Update*) để tối ưu hóa hiệu năng xử lý.

1. Tham số điều hướng chiến thuật:

- α (Alpha):

Đại diện cho giá trị tốt nhất (lớn nhất) mà người chơi MAX (Máy) có thể đảm bảo đạt được trên lộ trình từ nút gốc đến nút hiện tại.

Giá trị này được khởi tạo bằng âm vô cực:

$$-\infty$$

và được cập nhật liên tục khi tìm thấy các nhánh có lợi thế cao hơn.

- β (Beta):

Đại diện cho giá trị tốt nhất (nhỏ nhất) mà người chơi MIN (Người chơi) có thể đảm bảo đạt được.

Giá trị này được khởi tạo bằng dương vô cực:

$$+\infty$$

và giảm dần khi phát hiện các nhánh gây bất lợi cho MAX.

2. Cơ chế cập nhật điểm số cộng dồn (*Incremental Update*):

Trong quá trình duyệt cây, để tiết kiệm việc tính toán lại toàn bộ bàn cờ tại các nút lá ($\text{depth} = 0$), thuật toán duy trì và cập nhật biến điểm số xuyên suốt theo quy trình:

(a) Lưu trữ điểm cục bộ cũ:

Trước khi thực hiện một nước đi giả định (r, c) , hệ thống gọi hàm `get_local_score(r, c)` để ghi nhận giá trị điểm số của các dải ô chịu ảnh hưởng trực tiếp bởi vị trí đó.

(b) Cập nhật tổng điểm:

Sau khi đặt quân cờ thử nghiệm, điểm cục bộ mới tại (r, c) được tính toán lại. Tổng điểm của toàn bộ bàn cờ (`self.current_board_score`) được cập nhật ngay lập tức bằng cách cộng thêm phần chênh lệch:

$$\text{score} = \text{score} + (\text{new_score} - \text{old_score})$$

(c) **Truy xuất giá trị tại nút lá:**

Khi đạt giới hạn độ sâu, AI chỉ cần trả về giá trị hiện hành của biến điểm số. Quy trình này đưa độ phức tạp của hàm lượng giá từ:

$$O(N^2)$$

về mức hằng số:

$$O(1)$$

(d) **Hoàn tác trạng thái (Backtrack):**

Sau mỗi lần đệ quy, hệ thống thực hiện trừ lại phần chênh lệch điểm và thu hồi quân cờ để trả bàn cờ về trạng thái gốc, đảm bảo tính khách quan cho các nhánh duyệt sau.

3. **Tối ưu hóa thứ tự duyệt (*Move Ordering*):**

Thuật toán sử dụng danh sách nước đi đã được sắp xếp thông qua hàm `get_ordered_moves()`.

Việc ưu tiên duyệt các nước đi có điểm số tiềm năng cao giúp các ngưỡng α và β đạt được giá trị bão hòa nhanh chóng, tạo tiền đề cho việc cắt bỏ các nhánh dư thừa ở giai đoạn sớm của quá trình tìm kiếm.

7.3 Cơ chế cắt nhánh (Pruning)

Trong quá trình duyệt qua danh sách các nước đi hợp lệ:

- **Tại nút MAX:** Sau khi duyệt xong một nhánh con, **alpha** được cập nhật bằng giá trị lớn nhất giữa **alpha** hiện tại và điểm số vừa tính được. Nếu tại bất kỳ thời điểm nào, **beta** $\leq$ **alpha**, vòng lặp duyệt nước đi sẽ lập tức bị ngắt (**break**). Nhánh này bị cắt bỏ vì người chơi MIN ở phía trên chắc chắn sẽ không bao giờ cho phép người chơi MAX đi vào nhánh này.
- **Tại nút MIN:** Tương tự, **beta** được cập nhật bằng giá trị nhỏ nhất giữa **beta** hiện tại và điểm số tính được. Nếu **beta** $\leq$ **alpha**, vòng lặp cũng bị ngắt.

7.4 Kỹ thuật sắp xếp nước đi (Move Ordering)

Hiệu suất của thuật toán cắt tỉa Alpha-Beta phụ thuộc rất lớn vào thứ tự duyệt các nhánh. Nếu thuật toán xem xét các nhánh "tốt nhất" trước, các giới hạn α và β sẽ được cập nhật

nhANH chóng, từ đó tạo ra nhiều cơ hội cắt bỏ (pruning) các nhánh không cần thiết ở giai đoạn sớm. Để hiện thực hóa điều này, chương trình triển khai hàm `get_ordered_moves()` với cơ chế hoạt động như sau:

- **Đánh giá sơ bộ:** Thay vì duyệt danh sách các nước đi hợp lệ (`valid_moves`) một cách ngẫu nhiên hoặc tuần tự, hệ thống sẽ thực hiện "đánh thử" (simulate) từng nước đi lên bàn cờ.
- **Chấm điểm nhanh:** Tại mỗi nước đi giả định, hàm `evaluate_board()` được gọi để tính toán điểm số tĩnh của thế cờ ngay tại thời điểm đó. Sau khi lưu trữ điểm số, hệ thống lập tức "hoàn tác" (undo) nước đi để khôi phục trạng thái bàn cờ ban đầu.
- **Sắp xếp chiến lược:** Danh sách các nước đi sau đó được sắp xếp (sort) dựa trên điểm số vừa tính được:
 - *Tại lượt của Máy (nút MAX - `is_maximizing = True`):* Danh sách được sắp xếp theo chiều giảm dần (`reverse=True`). Trí tuệ nhân tạo sẽ ưu tiên duyệt các nhánh mang lại điểm số đánh giá cao nhất trước.
 - *Tại lượt của Người chơi (nút MIN - `is_maximizing = False`):* Danh sách được sắp xếp theo chiều tăng dần, ưu tiên duyệt các nhánh có điểm số thấp nhất (gây bất lợi nhất cho Máy).

Hiệu quả thực nghiệm: Kỹ thuật Move Ordering chính là chìa khóa giải thích tốc độ xử lý vượt trội của Alpha-Beta ở những thế cờ phức tạp có không gian trạng thái bùng nổ. Điển hình tại **Trạng thái 5 (Trạng thái phân tán)**, ở Độ sâu 3, trong khi Minimax phải duyệt "mù quáng" tới 237,205 trạng thái, Alpha-Beta nhờ định hướng từ Move Ordering đã nhanh chóng tìm ra nhánh tối ưu để cắt tĩa, giảm số lượng trạng thái cần duyệt xuống chỉ còn 4,194 (tương đương mức giảm $\approx 98.2\%$).

8 Hàm đánh giá trạng thái

8.1 Kỹ thuật Cửa sổ trượt (Sliding Window)

Do AI không thể duyệt cây tìm kiếm đến khi ván cờ kết thúc, hàm `evaluate_board()` đóng vai trò như "trực giác" của máy tính. Nó định lượng mức độ có lợi của một thế cờ để đưa ra quyết định khi chạm giới hạn độ sâu.

Phương pháp được sử dụng là phân tích **Cửa sổ trượt kích thước 4 ô**. Thuật toán sẽ trượt một khung 4 ô liên tiếp đi qua toàn bộ bàn cờ theo 4 hướng:

1. Hàng ngang.
2. Hàng dọc.
3. Đường chéo chính ($\backslash$).
4. Đường chéo phụ ($/$).

8.2 Hệ thống tính điểm (Heuristic Scoring)

Tại mỗi "cửa sổ", hàm `evaluate_window()` sẽ đếm số lượng quân của Máy (0), quân của Người chơi (X) và số ô trống ('.'), từ đó cộng hoặc trừ điểm cho bàn cờ theo quy tắc:

Đánh giá Tấn công (Lợi thế của Máy):

- 4 quân 0: +100,000 điểm (Chiến thắng).
- 3 quân 0 + 1 ô trống: +5,000 điểm.
- 2 quân 0 + 2 ô trống: +100 điểm.
- 1 quân 0 + 3 ô trống: +10 điểm.

Đánh giá Phòng thủ (Lợi thế của Người chơi):

- 4 quân X: -100,000 điểm (Thua cuộc).
- 3 quân X + 1 ô trống: -10,000 điểm.
- 2 quân X + 2 ô trống: -150 điểm.
- 1 quân X + 3 ô trống: -15 điểm.

8.3 Chiến thuật ưu tiên phòng thủ

Một điểm nhấn quan trọng trong thiết kế hàm lượng giá là việc áp dụng chiến lược đánh giá bất đối xứng (*Asymmetric Scoring*), trong đó hệ số phạt dành cho các thế cờ phòng thủ luôn được thiết lập với giá trị tuyệt đối lớn hơn hệ số thưởng của các thế cờ tấn công ở cùng một quy mô đe dọa.

Logic này định hình trực tiếp “tính cách” và hành vi chiến thuật của trí tuệ nhân tạo trên bàn cờ.

Cơ chế chênh lệch điểm số

Hệ thống lượng giá phân bổ các trọng số một cách có chủ đích dựa trên số lượng quân cờ liên tiếp và độ thoáng của dải ô. Cụ thể:

- **Ở mức độ đe dọa cao (3 quân cờ):**

Khi Máy (AI) có 3 quân cờ và 1 ô trống, hệ thống chỉ cộng thêm:

$$+5,000$$

điểm.

Tuy nhiên, nếu Người chơi có 3 quân cờ và 1 ô trống, hệ thống lập tức trừ:

$$-10,000$$

điểm.

- **Ở mức độ đe dọa trung bình (2 quân cờ):**

Máy tạo được chuỗi 2 quân và 2 ô trống nhận điểm thưởng:

$$+100$$

trong khi chuỗi tương tự của Người chơi mang lại mức phạt:

$$-150$$

- **Ở mức độ khởi đầu (1 quân cờ):**

Mức chênh lệch duy trì ở tỷ lệ:

$$+10$$

điểm cho tấn công và:

$$-15$$

điểm cho phòng thủ.

Tác động toán học lên thuật toán tìm kiếm

Trong cây quyết định Minimax và Alpha-Beta, mục tiêu tối thượng của Máy (nút MAX) là tối đa hóa điểm số tổng thể. Việc thiết lập hệ số phạt vượt trội tạo ra một áp lực toán học ép buộc AI phải phản ứng với các rủi ro.

Giả sử trên bàn cờ đang tồn tại đồng thời hai tình huống:

- AI có cơ hội nối thành chuỗi 4 quân (từ chuỗi 3 quân hiện tại).
- Người chơi cũng có một chuỗi 3 quân rộng 1 đầu cần được chặn.

Nếu AI chọn tấn công để lấy:

$$+5,000$$

điểm mà bỏ qua chuỗi của Người chơi, ở độ sâu tiếp theo, điểm số trả về sẽ lập tức bị kéo tụt xuống âm sâu:

$$-10,000$$

điểm hoặc thậm chí:

$$-1,000,000$$

điểm nếu Người chơi hoàn thành chuỗi 4 quân.

Theo nguyên lý tối thiểu hóa rủi ro, AI sẽ đánh giá nhánh tấn công mù quáng là một nhánh cực kỳ bất lợi. Từ đó, thuật toán sẽ chọn nhánh nước đi đặt quân vào vị trí đánh chặn chuỗi của đối thủ.

Hành động này giúp triệt tiêu rủi ro bị trừ điểm nặng, đồng thời bảo toàn được điểm số hiện tại của hệ thống.

Ý nghĩa chiến thuật thực tiễn

Cấu trúc điểm số bất đối xứng này giúp khắc phục triệt để nhược điểm “tham lam” (*Greedy*) thường thấy ở các AI cờ Caro cơ bản.

Thay vì mãi mê xây dựng chuỗi quân của riêng mình và bỏ qua các rủi ro nhãn tiền, AI được lập trình để ưu tiên sự an toàn. Hành vi “phòng ngự chủ động” này đảm bảo

Máy sẽ liên tục phong tỏa các hướng phát triển thế trận của đối phương, triệt tiêu mọi rủi ro tiềm tàng trước khi tìm kiếm cơ hội phản công, tạo ra một đối thủ cực kỳ vững chắc và khó bị đánh bại trong thực tế.

9 Thiết kế các trạng thái thử nghiệm

Để đánh giá toàn diện hiệu năng và chất lượng ra quyết định của thuật toán, nhóm đã thiết lập 5 trạng thái bàn cờ với các đặc điểm chiến thuật khác nhau:

1. **Đầu ván (Trống):** Bàn cờ chưa có quân nào. Đánh giá tốc độ xử lý khi không gian trạng thái ở mức lớn nhất (toàn bộ khu vực trung tâm).
2. **Người chơi sắp thắng (Cần chặn):** Người chơi (X) đã có 3 quân liên tiếp và rỗng 2 đầu. Đánh giá khả năng ưu tiên phòng ngự của AI.
3. **Máy có thể thắng ngay:** Máy (O) đã có 3 quân liên tiếp và rỗng 2 đầu. Đánh giá khả năng dứt điểm của AI.
4. **Giữa ván (Hai bên cùng tấn công):** Thế trận giằng co, nhiều cụm quân đan xen. Đánh giá khả năng cân bằng giữa tấn công và phòng thủ.
5. **Trạng thái phân tán (Nhiều nhánh):** Các quân cờ nằm rải rác khắp bàn cờ. Đánh giá sức chịu đựng của thuật toán khi hệ số nhánh (branching factor) tăng vọt.

10 Bảng kết quả thực nghiệm

Dưới đây là số liệu ghi nhận được khi chạy hai thuật toán Minimax và Alpha-Beta trên 5 trạng thái đã thiết kế ở các độ sâu (Depth) từ 1 đến 4. Tại độ sâu 4, thuật toán Minimax bị buộc dừng (Timeout) do hiện tượng bùng nổ không gian trạng thái.

Bảng 2: So sánh hiệu năng giữa Minimax và Alpha-Beta

Trạng thái	Thuật toán	Độ sâu	Nước đi	Số TT đã xét	Thời gian (s)
1. Đầu ván (Trống)	Minimax	1	(4,4)	2	0.0006
	Alpha-Beta	1	(4,4)	2	0.0004
	Minimax	2	(4,4)	10	0.0013
	Alpha-Beta	2	(4,4)	10	0.0014
	Minimax	3	(4,4)	98	0.0104

Trạng thái	Thuật toán	Độ sâu	Nước đi	Số TT đã xét	Thời gian (s)
	Alpha-Beta	3	(4,4)	49	0.0068
	Minimax	4	N/A	> 500000	Timeout
	Alpha-Beta	4	(4,4)	323	0.0294
2. Người chơi sắp thắng (Cần chặn)	Minimax	1	(2,5)	18	0.0010
	Alpha-Beta	1	(2,5)	18	0.0010
	Minimax	2	(5,5)	342	0.0284
	Alpha-Beta	2	(5,5)	55	0.0081
	Minimax	3	(5,5)	6448	0.3864
	Alpha-Beta	3	(5,5)	74	0.0215
	Minimax	4	N/A	> 500000	Timeout
	Alpha-Beta	4	(5,5)	100	0.0211
3. Máy có thể thắng ngay	Minimax	1	(1,5)	17	0.0016
	Alpha-Beta	1	(1,5)	17	0.0015
	Minimax	2	(1,5)	258	0.0224
	Alpha-Beta	2	(1,5)	31	0.0084
	Minimax	3	(1,5)	4735	0.2413
	Alpha-Beta	3	(1,5)	280	0.0263
	Minimax	4	N/A	> 500000	Timeout
	Alpha-Beta	4	(1,5)	501	0.1172
4. Giữa ván (Hai bên cùng tấn công)	Minimax	1	(4,5)	19	0.0010
	Alpha-Beta	1	(4,5)	19	0.0010
	Minimax	2	(4,5)	370	0.0209
	Alpha-Beta	2	(4,5)	54	0.0096
	Minimax	3	(4,5)	7754	0.3549
	Alpha-Beta	3	(4,5)	468	0.0326
	Minimax	4	N/A	> 500000	Timeout
	Alpha-Beta	4	(4,5)	860	0.1914
5. Trạng thái phân tán (Nhiều nhánh)	Minimax	1	(7,3)	63	0.0052
	Alpha-Beta	1	(7,3)	63	0.0054
	Minimax	2	(3,3)	3881	0.1491
	Alpha-Beta	2	(3,3)	246	0.0604
	Minimax	3	(7,3)	237205	9.2865
	Alpha-Beta	3	(7,3)	4278	0.2881
	Minimax	4	N/A	> 500000	Timeout
	Alpha-Beta	4	(7,3)	8334	3.6598

11 Nhận xét về kết quả thực nghiệm

11.1 Tính chính xác của Alpha-Beta

Dữ liệu từ Bảng 2 minh chứng rõ ràng: Ở cùng một độ sâu ($\text{Depth} = 1, 2, 3$), Alpha-Beta luôn trả về kết quả nước đi **giống hệt 100%** so với thuật toán Minimax thuần túy. Điều này khẳng định cơ chế cắt nhánh $\alpha - \beta$ hoàn toàn không làm mất mát thông tin hay thay đổi bản chất tối ưu của cây quyết định.

11.2 Khả năng cắt giảm trạng thái (Nodes Visited)

Alpha-Beta thể hiện sức mạnh vượt trội trong việc loại bỏ các nhánh không cần thiết, đặc biệt ở các thể cờ phức tạp:

- Ở **Trạng thái 4 (Depth 3)**, Minimax phải duyệt 7,754 node, trong khi Alpha-Beta chỉ xét 461 node (giảm $\approx 94\%$).
- Đặc biệt tại **Trạng thái 5 (Depth 3)**, Minimax mất kiểm soát khi phải duyệt tới 237,205 node. Ngược lại, nhờ kết hợp kỹ thuật *Move Ordering* (đã cài đặt trong mã nguồn), Alpha-Beta sớm tìm ra nhánh cắt và chỉ phải duyệt 4,194 node (giảm $\approx 98.2\%$).

11.3 Ảnh hưởng của độ sâu đến thời gian chạy

Độ sâu tìm kiếm (*Depth*) tỷ lệ thuận với cấp số nhân của không gian trạng thái cần duyệt. Qua các kịch bản kiểm thử, đặc biệt sau khi tích hợp kỹ thuật lượng giá cộng dồn (*Incremental Update*), sự phân hóa về thời gian chạy giữa các thuật toán và độ sâu được thể hiện một cách sâu sắc.

Đối với Minimax thuần túy

Dù hàm lượng giá đã được tối ưu độ phức tạp xuống:

$$O(1)$$

thuật toán này vẫn không thể vượt qua giới hạn toán học của sự bùng nổ tổ hợp:

$$O(b^d)$$

Ở các thế cờ phân tán (Trạng thái 5), mặc dù thời gian chạy ở Độ sâu 2 và Độ sâu 3 có sự cải thiện so với trước tối ưu (trước đó hệ thống mất 0.72 giây cho Độ sâu 2 và 45.7 giây cho Độ sâu 3), nhưng khi tiến lên Độ sâu 4, hệ thống vẫn hoàn toàn bị treo (*Timeout*).

Nguyên nhân cốt lõi là do số lượng trạng thái phải duyệt vẫn giữ nguyên ở mức khổng lồ:

$$> 500,000 \text{ nút}$$

Việc giảm thiểu thời gian tính toán trên từng nút riêng lẻ không đủ để bù đắp lại khối lượng nút sinh ra quá lớn theo cấp số nhân.

Đối với Alpha-Beta tích hợp Incremental Update

Đây là khu vực mà sự tối ưu phát huy sức mạnh tối đa, triệt tiêu hoàn toàn nút thắt cổ chai về hiệu năng tính toán.

- **Ở Độ sâu 1 và 2:**

Thời gian xử lý diễn ra gần như tức thời (tính bằng mili-giây), do thuật toán vừa cắt giảm được số lượng lớn nhánh duyệt, vừa không phải tốn chi phí tính toán lại điểm số cho toàn bộ 81 ô cờ.

- **Ở Độ sâu 3:**

Thời gian chạy duy trì ở mức cực thấp, cải thiện vượt bậc so với con số:

$$1.67 \text{ giây}$$

của phiên bản Alpha-Beta chưa được tối ưu lượng giá cục bộ.

- **Ở Độ sâu 4 (Mức Khó):**

Đây là bước nhảy vọt lớn nhất về mặt kỹ thuật.

Trước khi tối ưu, Alpha-Beta mất khoảng:

$$22.8 \text{ giây}$$

để hoàn thành Độ sâu 4 tại Trạng thái 5.

Nhờ sự cộng hưởng của ba nền tảng kỹ thuật:

- Cắt nhánh $\alpha - \beta$ (giảm tổng số nút)
- Sắp xếp nước đi định hướng (*Move Ordering* – tìm nhánh cắt tĩa sớm)
- Lượng giá cộng dồn (*Incremental Heuristic* – đưa chi phí đánh giá nút lá về $O(1)$)

thời gian xử lý đã được rút ngắn xuống ngưỡng thời gian thực (*real-time*).

Hệ thống giải quyết trơn tru khối lượng dữ liệu lớn, cho phép AI duy trì tầm nhìn chiến thuật sâu rộng mà không làm gián đoạn trải nghiệm thi đấu.

Nhìn chung, việc ứng dụng *Incremental Update* không làm thay đổi số lượng trạng thái mà cây tìm kiếm phải duyệt, nhưng nó làm giảm đột phá “chi phí thời gian” để xử lý từng trạng thái.

Sự can thiệp này biến Alpha-Beta từ một thuật toán tối ưu số lượng nút trở thành một hệ thống thông minh thực chiến, đáp ứng hoàn hảo yêu cầu khắt khe về tốc độ.

11.4 Ảnh hưởng của độ sâu đến chất lượng nước đi

Quan sát **Trạng thái 5**, ta thấy hiện tượng *Horizon Effect* (hiệu ứng chân trời). Ở Depth 1 và Depth 3, AI chọn tọa độ (7, 3), nhưng ở Depth 2 AI lại chọn (3, 3). Điều này cho thấy: Ở độ sâu thấp, AI chỉ đánh giá dựa trên giá trị tính của hàm lượng giá trước mắt (Greedy). Khi tăng độ sâu lên mức số chẵn (Depth 2), AI thấy được nước đi phản kháng của người chơi nên thay đổi quyết định để phòng thủ. Khi tiếp tục tăng lên Depth 3, AI lại thấy được "nước đi đáp trả của chính mình sau khi bị phản kháng", từ đó đưa ra quyết định tấn công sắc bén hơn.

12 Ưu điểm và Hướng cải tiến

12.1 Ưu điểm

Tối ưu hóa đa tầng toàn diện

Hệ thống không chỉ dừng lại ở việc cài đặt thuật toán Alpha-Beta cơ bản mà đã xây dựng thành công một chuỗi kỹ thuật tối ưu chuyên sâu và đồng bộ.

Cụ thể:

- Kỹ thuật *Adjacent Search* giúp giới hạn nghiêm ngặt không gian sinh nước đi (chỉ xét các ô lân cận quân cờ hiện có), làm giảm đáng kể hệ số nhánh:

b

- Kỹ thuật *Move Ordering* đóng vai trò định tuyến tìm kiếm, đánh giá cục bộ để ưu tiên duyệt các nước đi tiềm năng nhất, giúp các ngưỡng α và β bão hòa sớm nhằm cắt tỉa tối đa các nhánh thừa.
- Sự đột phá đến từ *Incremental Evaluation* (Cập nhật lượng giá cộng dồn), giúp hệ thống duy trì theo dõi điểm số bàn cờ bằng các hiệu số thay vì tính toán lại từ đầu.

Chuỗi kết hợp này đã giải phóng tối đa tài nguyên bộ nhớ và CPU, cho phép AI duy trì độ trễ cực thấp (chuẩn thời gian thực) ngay cả khi phải xử lý khối lượng trạng thái khổng lồ ở Độ sâu 4.

Cơ chế lượng giá sắc bén và thực dụng

Hàm đánh giá trạng thái đạt được độ phức tạp lý tưởng:

$O(1)$

tại mỗi nút lá.

Không chỉ nhanh, hàm lượng giá còn được thiết kế với cơ chế tính điểm bất đối xứng (ưu tiên phòng ngự) để tạo ra một thế trận chặt chẽ.

Hơn nữa, việc tích hợp cơ chế trừng phạt theo độ sâu (*Depth Penalty*) – cộng hoặc trừ thêm biến `depth` vào điểm số cực đại/cực tiểu – mang lại cho AI một “nhận thức về thời gian” rõ rệt.

Nhờ đó, AI biết cách:

- Duy trì sự sống lâu nhất có thể khi rơi vào thế cờ bất lợi (chọn con đường thua chậm nhất).
- Ra quyết định kết liễu trận đấu nhanh gọn nhất, dứt khoát nhất khi có cơ hội vây hãm đối thủ.

12.2 Hạn chế và Hướng cải tiến

Hạn chế của phương pháp lượng giá tuyến tính

Phương pháp *Cửa sổ trượt* (*Sliding Window*) hiện tại đo lường rất tốt sức mạnh của các chuỗi quân trên các đường thẳng độc lập (ngang, dọc, chéo), nhưng lại thiếu đi sự liên kết điểm số đa trục.

Điều này khiến AI gặp khó khăn cục bộ trong việc nhận diện sớm các bẫy chiến thuật phức tạp mang tính không gian, điển hình là thế “gài ngã ba” (tạo ra hai đường uy hiếp cùng lúc từ một nước đi).

AI đôi khi phản ứng chậm với các rủi ro tổng thể nếu không được duyệt đủ sâu để nhìn thấy điểm kết thúc của các đường đe dọa đó.

Hướng cải tiến 1 – Tích hợp bộ nhớ trạng thái (Zobrist Hashing)

Đặc thù của trò chơi Caro là sự bùng nổ của các trạng thái trùng lặp (*Transpositions*).

Một thế cờ giống hệt nhau có thể được tạo ra từ nhiều thứ tự đặt quân khác nhau. Việc tiếp tục duyệt lại các nhánh đã tính toán là một sự lãng phí tài nguyên nghiêm trọng.

Giải pháp tối ưu tiếp theo là tích hợp Bảng băm (*Transposition Table*) thông qua kỹ thuật mã hóa *Zobrist Hashing*.

Khi đó, cây tìm kiếm sẽ được chuyển hóa thành đồ thị có hướng không chu trình (*DAG*).

Mỗi thế cờ khi đánh giá xong sẽ được lưu trữ lại; nếu gặp lại cấu hình này ở các nhánh khác, AI chỉ việc truy xuất kết quả từ bộ nhớ, giúp đẩy tốc độ thuật toán lên mức giới hạn tối đa của phần cứng.

Hướng cải tiến 2 – Xây dựng Thư viện khai cuộc (Opening Book)

Ở giai đoạn khai cuộc (3 đến 5 nước đi đầu tiên), không gian bàn cờ trống trải khiến việc sử dụng Minimax hay Alpha-Beta để đệ quy sâu là hoàn toàn lãng phí và thiếu cơ sở lượng giá vững chắc.

Việc tích hợp một *Thư viện khai cuộc* (*Opening Book*) chứa các biến thể phát triển quân cờ chuẩn mực của cờ Caro quốc tế (như các thế sao băng, thế trắng khuyết, ...) sẽ khắc phục triệt để vấn đề này.

Hệ thống sẽ bỏ qua bước tìm kiếm đệ quy và phản hồi tức thời dựa trên dữ liệu tra cứu, qua đó bảo toàn toàn bộ tài nguyên tính toán cho giai đoạn giữa và cuối ván đấu.